

Q LFX

@



**LFX Mentorship Proposal:
CNCF Kmesh (2026 Term 3)**

**[Develop MCP Server for AI-Native
Kmesh Service Mesh Management]**

Letter to the Program & Mentors

1. How did you find out about our mentorship program?

I regularly browse through the LFX Mentorship portal to see what's new in the CNCF ecosystem. I stumbled across Kmesh while reading about how eBPF can eliminate sidecar proxy bloat, and I was genuinely hooked by the architecture. When Issue #1800 dropped asking for an MCP server, it felt like the perfect match, I've already spent time building AI tool-calling systems, and I've been contributing to Kmesh over the past month. It was an immediate no-brainer to apply

2. Why are you interested in this program?

Kmesh solves the sidecar performance problem brilliantly, but inspecting low-level eBPF state and Waypoint proxies is still tough to do on the fly. An MCP server brings AI-native troubleshooting to Kmesh, allowing agents like Claude Desktop and Cursor to inspect mesh state in plain English. I'm excited about this project because it solves a real operational headache, and I'd love the opportunity to build it this term.

3. What applicable experience and skills do you bring?

- *Active Kmesh Contributions:* Submitted 3 PRs (#1722, #1727, #1737) covering controller bugs, kmeshctl auto-detection, and documentation.
- *Agentic AI Infrastructure:* Built mdfmt, an AI documentation agent powered by Llama 3.3-70B with structured tool calling and context routing.

- *Security & Systems Stack*. Developed PNGProtect (adversarial security tool). Proficient in Go, Kubernetes, Docker, eBPF concepts, Protobuf/gRPC, and C/Rust fundamentals.
- *Protocol Mastery*: Researched the MCP 2026-07-28 specification, designing Streamable HTTP and Stdio transports while addressing prerequisites like Issue #1830.

4. What do you hope to get out of this mentorship experience?

- Get real feedback from Yash and Sumit to become a much better Go engineer and learn how maintainers review and test code.
- Build a working MCP server that people actually use when troubleshooting Kmesh.
- Stick around in the Kmesh community and keep contributing even after the mentorship ends.

Basic Information:

- **Target Organization:** Cloud Native Computing Foundation (CNCF) — Kmesh
- **Upstream Issue:** [#1800 - Develop MCP Server for AI-Native Kmesh Service Mesh Management](#)
- **LFX Project ID:** 267f9270-0d36-4999-bdf5-c942b95c8e7a
- **Mentors:** Yash Israni ([@yashisrani](#))
Sumit Goyal ([@itvi-1234](#))
- **Applicant Name:** Apurve Karanwal
- **Email:** apurvekaranwal282@gmail.com
- **GitHub:** <https://github.com/ApurveKaranwal>
- **LinkedIn:** <https://www.linkedin.com/in/apurvekaranwal>

About me:

I am a 20-year-old Computer Science undergraduate passionate about low-level cloud-native systems, kernel networking, and agentic AI infrastructure. My technical stack includes Go, C, Rust fundamentals, Kubernetes, Docker, eBPF concepts, Protobuf, gRPC, and the Model Context Protocol (MCP). I'm aware that Kmesh's Waypoint proxies rely on Orion, its Rust-based Envoy-alternative proxy, and while my current Rust exposure is at the fundamentals level, I've scoped direct Orion-level introspection as a natural post-mentorship follow-on (see Non-Goals) as that experience deepens. My relevant experience includes:

- [mdfmt](#): Built an intelligent documentation agent using Llama 3.3-70B, React 19, and Express 5, with structured tool calling and context routing.
- [PNGProtect](#): Developed a security tool for detecting and mitigating adversarial payload injection threats.
- **Kmesh**: Actively contributing to kmesh-net/kmesh, with 3 open pull requests (#1722, #1727, #1737) covering controller logic, kmeshctl CLI auto-detection, and documentation, currently under maintainer review.

I am particularly interested in Kmesh's eBPF-based networking and the opportunity to combine low-level mesh observability with MCP-based tool calling, enabling AI agents to reason about mesh state and assist with debugging and operations.

Contributions to kmesh:

<u>Repository</u>	<u>PR</u>	<u>Description</u>
kmesh-net/kmesh	#1722 (open)	Fixes controller log bugs, CNI variable shadowing, and quote syntax errors.
kmesh-net/kmesh	#1727 (open)	Unifies kmeshctl pod auto-detection and fixes waypoint status latency.
kmesh-net/kmesh	#1737 (open)	Fixes typos, incorrect CLI flags, and formatting in docs.

What is kmesh?

Kmesh is an eBPF-based, sidecarless service mesh data plane for Kubernetes hosted in the CNCF. Kmesh moves Layer 4 load balancing and Layer 7 HTTP traffic governance directly into the Linux kernel using eBPF, eliminating sidecar proxy containers to reduce network latency by 60% and resource consumption by 70%. Kmesh was initially incubated in the openEuler community and is actively developed by cloud-native community contributors.

Problem Statement & Motivation:

Diagnosing Kmesh connectivity issues currently requires engineers to manually inspect daemon debug endpoints, parse Envoy xDS protobuf dumps, and correlate them with kernel eBPF state such as sockmaps. This workflow is time-consuming and difficult to automate.

Issue #1800 proposes a dedicated MCP server under mcp/ that exposes Kmesh inspection and management capabilities as structured tools. This would allow AI agents such as Claude Code, Cursor, and GitHub Copilot to investigate and manage Kmesh through natural-language requests without requiring users to manually interpret raw xDS data or complex kmeshctl commands.

Goals

1. Support Stdio transport for local IDE integrations via kmeshctl mcp serve (accelerated stretch goal to de-risk tool schemas early) and Streamable HTTP for containerized deployments, superseding the SSE transport referenced in #1800 per the MCP 2026-07-28 deprecation

2. Official SDK & Protocol Compliance:

The server will be built using the official Tier 1 [Go SDK](#) operating in stateless mode per the latest MCP specification (2026-07-28). Utilizing stateless execution eliminates persistent session/sticky-routing overhead, enabling standard Kubernetes

Service load balancers to route Streamable HTTP requests natively without session dropouts.

3. Implement **10 core inspection tools** covering workloads, services, Envoy xDS state, XDP authorization, telemetry, logging, IPsec, and Waypoint proxies.
4. Build a flagship `diagnose_connectivity` tool for automated, multi-layer connectivity analysis.
5. Implement **5 guarded mutation tools** with `--enable-write`, dry-run previews, and mandatory confirm: true safeguards.
6. Quality & CI: Implement three Makefile targets (`make mcp-server`, `make mcp-test`, `make mcp-image`) covering four comprehensive testing tiers:
 - Unit Tests: >80% code coverage using gomock mocks.
 - Integration Tests: Handler-to-daemon HTTP port-forwarding tests.
 - End-to-End (E2E) Tests: Automated Kind cluster validation harness.
 - MCP Protocol Compliance Tests: Schema validation of `tools/list` and `tools/call` JSON-RPC frames against official MCP JSON schemas.
7. Ship the RBAC manifests (`ServiceAccount`, `ClusterRole`, `Role/RoleBindings`) for the ``kmesh-mcp-server`` identity as part of the deployment artifacts, alongside the Dockerfile.

Non-Goals

- Supporting non-Kubernetes bare-metal Kmesh deployments.
- Multi-cluster or cross-mesh federation; the MCP server targets a single Kmesh-managed cluster per deployment.
- Modifying Kmesh's underlying eBPF programs or xDS control-plane protocols.

Rust/Orion Proxy Telemetry:

Waypoint tools focus on Gateway API CRDs; direct Rust/Orion proxy introspection is scoped for post-mentorship.

Prerequisite & Technical Dependency (Issue #1830 Integration)

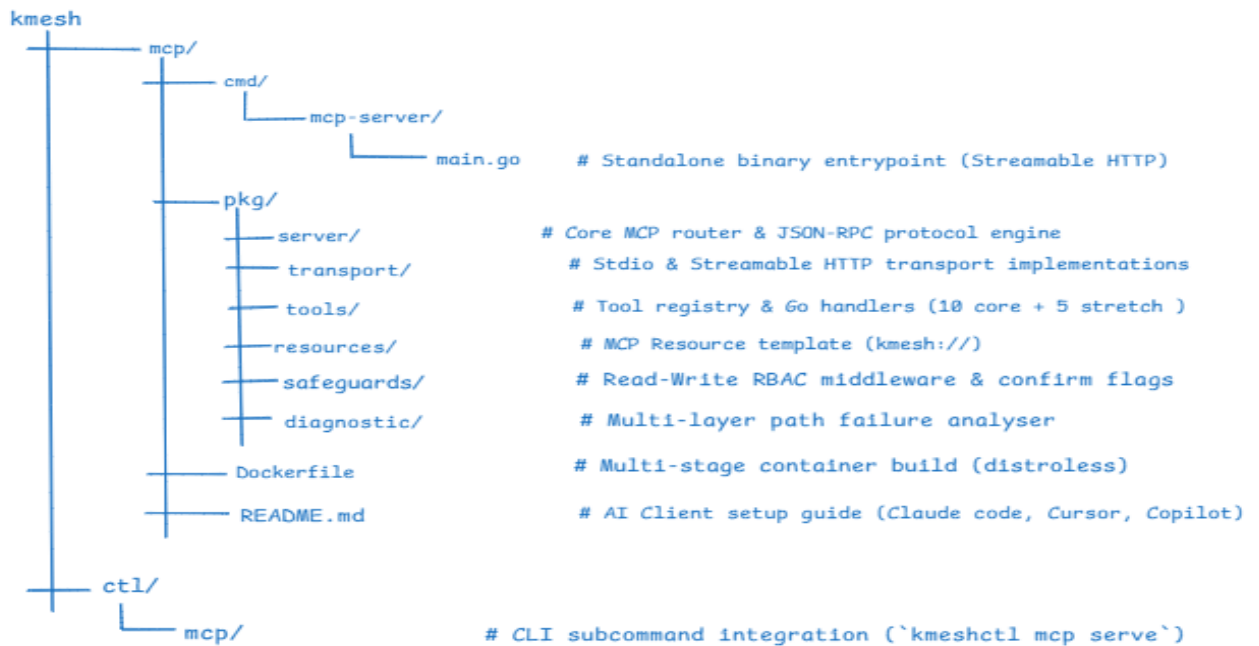
A critical technical prerequisite for the Stdio transport channel, (built early as an accelerated stretch goal), is addressing upstream [Issue #1830](#) ("Making kmeshctl output programmable").

Currently, kmeshctl subcommands write unstructured log strings directly to os.Stdout. Over Stdio transport, unformatted stdout writes pollute the raw JSON-RPC 2.0 stream, causing connected AI client agents (Cursor, Claude Desktop) to crash immediately.

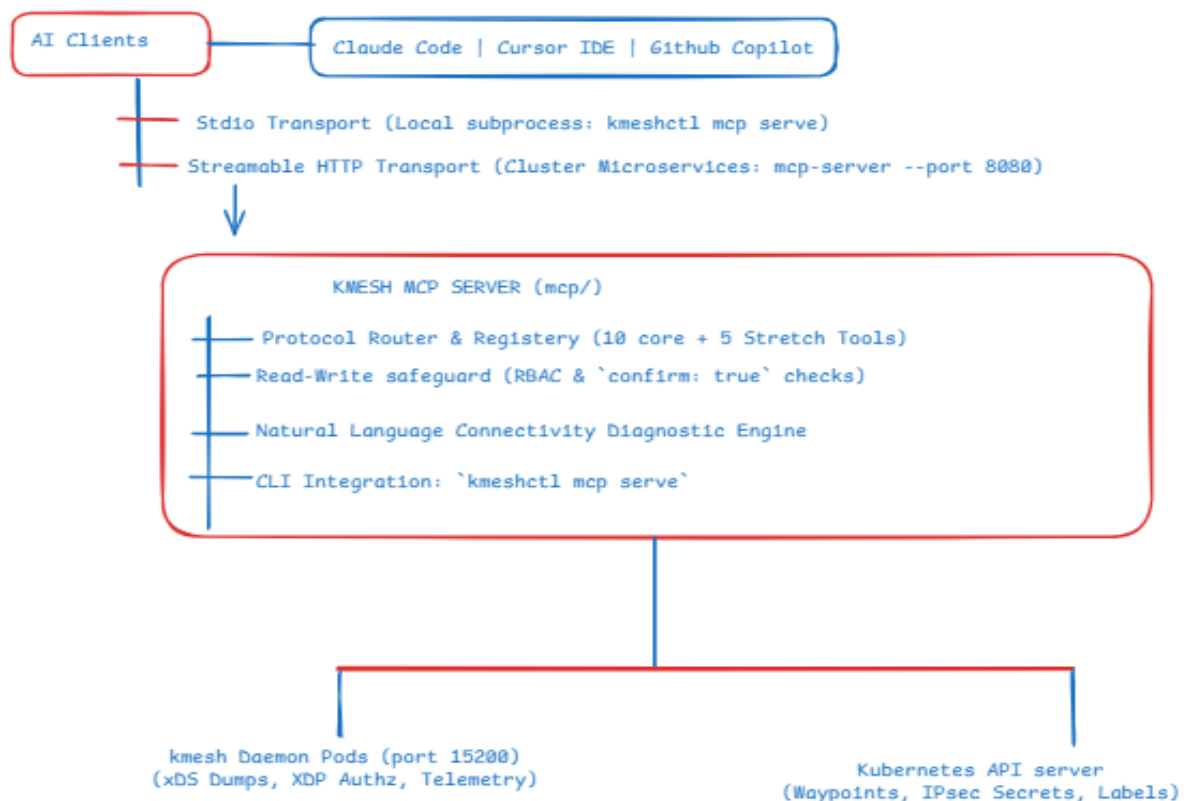
As part of Phase 1 bonding, I will ensure stdout is strictly reserved for JSON-RPC frame serialization, redirecting internal diagnostic logs and errors to os.Stderr or structured loggers.

Architecture:

A. Repository Directory Structure



B. Component Architecture Flow:



C. Modular Core Architecture (mcp/pkg/):

Core MCP logic lives in dedicated subpackages under ``mcp/pkg/``, shared by both the standalone server (``mcp/cmd/mcp-server``) and the CLI integration (``kmeshctl mcp serve``), so both surfaces run identical inspection and management logic.

Key subpackages:

Registry: dynamic tool registration (``tools/list``, ``tools/call``), so the catalog scales as Kmesh evolves.

Domain: typed ``ServerMode`` enums, base interfaces, JSON schemas, and tool contexts for consistent logic and mock testing (``gomonkey``).

Core: daemon interaction logic: pooled port forwarding to Port 15200, xDS dump parsing, XDP authz queries, Waypoint inspection, and path diagnostics.

``mcp/cmd/`` compiles the standalone Streamable HTTP server;
``ctl/mcp/`` wires ``kmeshctl mcp serve`` over Stdio. Shared code in ``mcp/pkg/`` keeps both paths in sync and simplifies hitting >80% test coverage.

Detailed Tool Specifications

I plan to deliver 15 MCP tools in total (10 core inspection tools + 5 stretch mutation tools), divided into four functional groups:

1. Config Dump & Workload Inspection Tools (4 Core Tools)

- list_services: Returns all Kubernetes services running in the Kmesh data plane, hostnames, VIPs, and active dataplane mode (Kernel-native vs. Dual-engine).
- get_workloads: Lists active workload pods managed by Kmesh, including IP addresses, protocols (HTTP/gRPC/TCP), and status.
- get_config_dump: Retrieves parsed Envoy xDS configuration dumps (listeners, routes, clusters) or dual-engine workloads from a Kmesh daemon pod via Port 15200.
- get_version_info: Retrieves Kmesh daemon build version, eBPF module status, and kernel version from /version on Port 15200.

2. Telemetry & Security Management Tools (4 Core + 3 Stretch Tools)

- get_authz_status (Core) & manage_authz (Stretch): Queries or toggles XDP authorization offloading on Kmesh daemons (/authz).
- get_monitoring_status (Core) & manage_monitoring (Stretch): Queries or toggles access logs, workload metrics, and connection metrics (/monitoring, /accesslog, /workload_metrics, /connection_metrics).
- get_daemon_logs (Core) & manage_logger_level (Stretch): Inspects logger scopes and sets log levels (e.g., default:debug) (/debug/loggers).
- get_ipsec_secret (Core): Inspects IPsec encryption configuration secret (kmesh-ipsec), AEAD algorithm details, and SPI key index.

3. Waypoint Proxy Tools (1 Core + 2 Stretch Tools)

- get_waypoint_status (Core): Lists Waypoint proxies in a namespace and reports Gateway API Programmed condition status.
- apply_waypoint & delete_waypoint (Stretch): Generates, applies, or removes Gateway API Waypoint CRDs with dry-run verification.

4. Natural Language Diagnostic Engine (1 Core Tool)

- diagnose_connectivity (Core): Performs multi-layer root-cause analysis across namespace labels (istio.io/dataplane-mode=Kmesh), eBPF XDP authorization maps, xDS routing tables, and Waypoint proxies.

[Natural Language Query: "Why is frontend failing to reach payment-service?"]



```
1. Check Namespace Dataplane Label (istio.io/dataplane-mode=Kmesh)
↳ If missing: Return "Namespace not enrolled in Kmesh data plane"

2. Query XDP Authorization eBPF Maps (/authz) on Source Pod Node
↳ If denied: Return "XDP eBPF filter dropping packets at socket"

3. Inspect xDS Route & Cluster Assignments (/debug/config_dump)
↳ If missing cluster/endpoint: Return "xDS route missing target"

4. Evaluate Waypoint Gateway API Readiness (gateway.Gateway)
↳ If GatewayConditionProgrammed != True: Return "Waypoint unready"
```



[Structured Root Cause Analysis + Actionable Remediation Command Output]

Transport Comparison (STDIO vs Streamable HTTP):

<u>Feature</u>	<u>STDIO Transport</u>	<u>Streamable HTTP Transport (MCP 2026-07-28 Spec)</u>
1) Primary use case	1) local IDE subprocess (Cursor, Claude)	1) Cluster-internal
2) Command	2) kmeshctl mcp serve --transport stdio	2) mcp-server --transport http --port 8080
3) Latency & Speed	3) Extremely fast, zero network overhead	3) Low network streaming latency (Single HTTP endpoint)
4) Security Risk	4) Isolated local process communication	4) Requires network security / auth
5) Selection	5) Prioritized early (stretch goal) for local IDE debugging	5) Recommended for remote AI microservices

Architectural Note on Transport Order:

While Streamable HTTP is the core transport, I'm building Stdio transport (a stretch-goal item), early to provide a local IDE debugging harness (Cursor, Claude Desktop). This de-risks tool handler schemas before they're wrapped in Streamable HTTP, sharing the same registry at low added cost.

Security & Authentication Model

Because mutation tools modify live cluster policy, security is enforced across three layers:

1. Identity & RBAC Boundaries:

- Stdio Mode (Local CLI): Inherits the invoking developer's kubeconfig directly.
- Streamable HTTP Mode (Cluster Microservice): Runs under a dedicated ServiceAccount (kmesh-mcp-server) with a scoped read-only ClusterRole for inspection and namespaced Role bindings for Gateway API CRDs.

2. Guarded Read-Write Mode (--enable-write):

- Operates in strict Read-Only Mode (ServerModeReadOnly) by default.
- Mutation tools require the explicit --enable-write flag at launch and a mandatory confirm: true payload parameter; otherwise, the server returns a Dry-Run Diff Preview.

3. Authentication & Protocol Alignment:

- Streamable HTTP uses Authorization headers (Bearer <token>) validated against the Kubernetes TokenReview API to authenticate callers as real K8s identities. While MCP 2026-07-28 defines OAuth 2.1 for remote clients, TokenReview provides zero-dependency, K8s-native authentication for cluster deployments.

Proof of Concept (PoC) & Implementation

Snippets:

To demonstrate technical feasibility, I have authored initial Proof of Concept (PoC) Go modules utilizing the architecture patterns defined by the official [Go SDK](#).

The core MCP server router is implemented in ``mcp/pkg/server/server.go``. This module initializes the central tool registry, registers default Kmesh inspection tools, and handles incoming JSON-RPC 2.0 requests over the Stdio transport channel. Below is an excerpt illustrating the core server structure:

```
~GO server.go U X
~GO server.go
1  package server
2
3  import (
4      "context"
5      "kmesh.net/kmesh/mcp/pkg/registry"
6      "kmesh.net/kmesh/mcp/pkg/transport"
7  )
8
9  type ServerMode int
10
11  const (
12      ServerModeReadOnly ServerMode = iota
13      ServerModeReadWrite
14  )
15
16  type Server struct {
17      registry *registry.ToolRegistry
18      mode     ServerMode
19  }
20
21  func NewServer(mode ServerMode) *Server {
22      s := &Server{
23          registry: registry.NewToolRegistry(),
24          mode:     mode,
25      }
26      s.registerDefaultTools()
27      return s
28  }
29
30  func (s *Server) IsWriteEnabled() bool {
31      return s.mode == ServerModeReadWrite
32  }
33
34  func (s *Server) StartStdio(ctx context.Context) error {
35      t := transport.NewStdioTransport()
36      return t.Listen(ctx, s.registry)
37  }
```

Proof of Concept Implementation for ``server.go``

In the implementation above, `NewServer()` initializes a `Server` struct with a dynamic `ToolRegistry` and a typed `ServerMode` (`ServerModeReadOnly` or `ServerModeReadWrite`) rather than a raw string, exposing `IsWriteEnabled()` so mutation-tool handlers can check the safeguard before applying any change. `StartStdio()` binds the server to `transport.NewStdioTransport()`, enabling seamless subprocess communication with local IDEs like Cursor and Claude Desktop.

In addition to the core server router, the daemon client integration is implemented in `mcp/pkg/core/daemon_client.go`. This package encapsulates all communication between the MCP server and Kmesh daemon pods using `utils.CreateKmeshPortForwarder` to dynamically establish secure port-forwarding tunnels to Port 15200 on `kmesh-daemon` pods in the `kmesh-system` namespace.

```
~GO daemon_client.go U X
~GO daemon_client.go
1  package core
2  import (
3      "fmt"
4      "io"
5      "net/http"
6
7      "kmesh.net/kmesh/ctl/utils"
8      "kmesh.net/kmesh/pkg/kube"
9  )
10 type DaemonClient struct {
11     cliClient kube.CLIClient
12 }
13 func (c *DaemonClient) FetchDaemonEndpoint(ctx context.Context, podName, path string) ([]byte, error) {
14     fw, err := utils.CreateKmeshPortForwarder(c.cliClient, podName)
15     if err != nil {
16         return nil, fmt.Errorf("port forwarder creation failed: %v", err)
17     }
18     if err := fw.Start(); err != nil {
19         return nil, fmt.Errorf("port forwarder start failed: %v", err)
20     }
21     defer fw.Close()
22
23     ctx, cancel := context.WithTimeout(ctx, 5*time.Second)
24     defer cancel()
25
26     url := fmt.Sprintf("http://%s%s", fw.Address(), path)
27     req, err := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
28     if err != nil {
29         return nil, err
30     }
31     resp, err := http.DefaultClient.Do(req)
32     if err != nil {
33         return nil, err
34     }
35     defer resp.Body.Close()
36
37     if resp.StatusCode != http.StatusOK {
38         return nil, fmt.Errorf("daemon endpoint %s returned status %d", path, resp.StatusCode)
39     }
40     return io.ReadAll(resp.Body)
41 }
```

Proof of Concept Implementation for `daemon_client.go`

In the implementation above, `FetchDaemonEndpoint()` takes a target `podName` and HTTP endpoint `path` (such as `/debug/config_dump/kernel-native`, `/authz`, or `/monitoring`). It initializes a temporary port-forwarding session using `utils.CreateKmeshPortForwarder`, starts the port forwarder, and issues a context-scoped GET request with a 5-second timeout. The response status is checked before the body is read, so a non-200 response from the daemon surfaces as an error instead of being silently returned as valid data; `defer fw.Close()` ensures the tunnel is always torn down.

Illustrative JSON-RPC 2.0 Schema Exchange

Below is an illustrative JSON-RPC 2.0 payload exchange demonstrating the tool-call schema for `list_services` over Stdio transport, showing how the MCP server would format Kmesh daemon inspection data for AI clients. Field names are based on Kmesh's `/debug/config_dump` output and will be confirmed once the handler is implemented against a live daemon.

```
{ } jsonrpc.json 1, U X
{ } jsonrpc.json
1  JSON-RPC Request (from AI Client):
2  {
3    "jsonrpc": "2.0",
4    "id": 1,
5    "method": "tools/call",
6    "params": {
7      "name": "list_services",
8      "arguments": {
9        "namespace": "default"
10     }
11  }
12  }
13
14  JSON-RPC Response (from Kmesh MCP Server):
15  {
16    "jsonrpc": "2.0",
17    "id": 1,
18    "result": {
19      "content": [
20        {
21          "type": "text",
22          "text": "Service: payment-service | VIP: 10.96.142.18:8080 | Mode: kernel-native"
23        }
24      ]
25    }
26  }
```

Tentative Timeline:

Timespan	Duration	Activity & Tasks
Sept 7 - Sept 14	1 Week	• Community Bonding & Setup: Discuss architecture on issue #1800 with mentors Yash & Sumit; set up Kind cluster with eBPF headers; establish mcp/ package structure & Dockerfile.
Sept 15 – Oct 5	3 Weeks	• Build server router (mcp/pkg/server/), Streamable HTTP transport (http.go), and Stdio transport (stdio.go, accelerated stretch goal). • Implement --read-only/--enable-write safeguards and add make mcp-server target.
Oct 6- Oct 19	2 Weeks	• Implement 8 core inspection tools. Run make mcp-test covering Unit tests (>80% coverage), Integration tests, and MCP Protocol Compliance tests.
Oct 20 – Nov 2	2 Weeks	• Internal Midterm Checkpoint & Diagnostic Engine: Complete remaining 2 core tools (get_waypoint_status & diagnose_connectivity decision-tree analyzer). Aligns with the official LFX Midterm Mentee Evaluation on Oct 20.
Nov 3 – Nov 16	2 Weeks	• Stretch Tools & CLI Integration: Implement 5 mutation tools with confirm: true checks. Integrate kmeshctl mcp serve CLI subcommand, kmesh:// resource templates & make mcp-image.
Nov 17 – Nov 27	1.5 Weeks	• Docs, E2E & Final Evaluation: Author RBAC manifests/Helm charts and User/Developer guides (architecture + adding new tools). Run E2E tests on Kind, publish blog post on kmesh.net, record video demo, and present at Kmesh meeting. Final Evaluation.

Risk Mitigation & Contingency Strategy

- Buffer Scope:

The 5 mutation tools (manage_authz, manage_monitoring, manage_logger_level, apply_waypoint, delete_waypoint) are scoped as modular stretch goals. If unexpected edge-cases arise in eBPF/xDS parsing, these can be deferred without impacting delivery of the 10 core inspection tools and the diagnose_connectivity engine.

- Incremental Documentation:

Docs are written at the end of each feature block, not the final week — so by Nov 17 only polish, the blog post, and the demo video remain.

Commitments and Availability:

- Weekly Time Commitment: I will dedicate 4–5 hours per day on weekdays (~20–25 hours/week) and 6–7 hours on weekends, totaling 30–35 hours per week throughout the LFX Term 3 mentorship period (September 7 – November 27).
- Sync Meetings: I am fully available to attend weekly Kmesh community meetings (Thursdays at 16:00 UTC+8) and mentor sync calls with Yash Israni ([@yashisrani](#)) and Sumit Goyal ([@itvi-1234](#)).

- Maintainer Alignment: Active architectural discussion on upstream Issue #1800 with mentors Yash Israni ([@yashisrani](#)) and Sumit Goyal ([@itvi-1234](#)), aligning on transport protocols, package layout, and safeguard requirements.

Post Mentorship Goals:

Even after the LFX mentorship concludes, I plan to stay actively involved in Kmesh as a long-term open-source contributor, helping maintain the MCP server module, reviewing PRs, and expanding AI-native tooling as Kmesh evolves.

References & Resources:

1. Upstream Issue [#1800](#)
2. LFX Mentorship Project:
<https://mentorship.lfx.linuxfoundation.org/project/267f9270-0d36-4999-bdf5-c942b95c8e7a>
3. Kmesh Official Website: <https://kmesh.net/>
4. Model Context Protocol (MCP) Specification:
<https://modelcontextprotocol.io/>
5. Kmesh GitHub Repository: <https://github.com/kmesh-net/kmesh>

Thank you for reviewing my proposal! :)